

ThreatLens

Security Assessment Platform

World Monitor - Security Assessment Report

CONFIDENTIAL - FOR AUTHORIZED RECIPIENTS ONLY

Project:	World Monitor
Target:	https://github.com/koala73/worldmonitor
Report Date:	September 17, 2026
Methodology:	Static Analysis . Dynamic Testing . Manual Validation
Prepared By:	ThreatLens Automated Scanner v1.0
Classification:	Confidential

1. EXECUTIVE SUMMARY

Overall Security Posture: MEDIUM RISK

5

TOTAL FINDINGS

0

CRITICAL

0

HIGH

2

MEDIUM

2

LOW

This automated security assessment identified 5 potential security issue(s) across 3 security category/categories. Of these, 5 remain open and 0 have been resolved. Critical and High severity findings should be addressed immediately.

Assessment Date: September 17, 2026

Target:

2. SEVERITY BREAKDOWN

Severity	Count	% of Total	Status
CRITICAL	0	0%	-
HIGH	0	0%	-
MEDIUM	2	40%	CONFIRMED
LOW	2	40%	REJECTED
INFO	1	20%	REJECTED

3. ASSESSMENT SCOPE & TARGET

Project Name: World Monitor

Description:

4. METHODOLOGY

ThreatLens performs a multi-phase security assessment combining static source-code analysis, dynamic application testing against authorized local targets, and automated validation. All testing is conducted in a non-destructive manner against explicitly authorized targets only. No testing is performed against third-party infrastructure, production systems outside the defined scope, or without explicit authorization.

Static Analysis:

Source code, configuration files, dependency manifests, and secret patterns are scanned without executing the application.

Dynamic Testing:

HTTP security headers, API endpoints, XSS reflection, and injection vectors are tested against a local authorized instance.

Validation:

Each finding is evaluated for confidence (CONFIRMED / LIKELY / POSSIBLE / FALSE_POSITIVE) using scanner-specific validation logic.

Evidence Collection:	Request/response pairs, code snippets, and configuration excerpts are captured as structured evidence.
Remediation Tracking:	Findings progress through a defined lifecycle: DETECTED -> CONFIRMED -> REMEDIATION -> RETESTING -> RESOLVED.

5. FINDINGS

F01 Potential: MCP Proxy DNS Rebinding Race (TOCTOU) on Vercel

MEDIUM

POSS

Confirme

Location: api/mcp-proxy.ts:329

Description

The MCP proxy endpoint (api/mcp-proxy.ts) fetches arbitrary upstream MCP server URLs. Two DNS pre-resolution checks are performed (assertServerUrlSafe line 329, revalidateBeforeFetch line 368), but a TOCTOU gap exists between the second DNS check and the actual fetch() call. A code comment at line 360 explicitly acknowledges this limitation on Vercel Edge Runtime. Cross-origin redirects are blocked as an additional mitigation. Exploitation requires: (1) a paid Pro API key, (2) attacker DNS control with low-TTL records, and (3) precise timing between the second DNS check and fetch. The double-check significantly narrows the window. Classified as potential, not confirmed exploitable.

Remediation

1. Route MCP fetches through Railway relay with socket-level IP pinning. 2. Add hostname allowlist as defense-in-depth. 3. Log/alert on unexpected outbound hostnames. 4. Apply Cloudflare DoH pinning to MCP proxy DNS resolution.

Evidence

[CODE_SNIPPET] Code comment at api/mcp-proxy.ts:360 acknowledging TOCTOU limitation

```
api/mcp-proxy.ts lines 360-365 (verbatim):
// Vercel Edge fetch does not expose a Node-style lookup/socket hook,
// so there is no way to guarantee that the IP we validated is the IP
// fetch() ultimately connects to; a DNS answer can change between our
// resolve and fetch's own resolve.
assertServerUrlSafe() at line 329
revalidateBeforeFetch() at line 368 (best-effort second check)
```

F02 Dependency: image-size <=2.0.2 - DoS via Infinite Loop in Image

MEDIUM

POSS

Confirme

Location: package-lock.json:1

Description

image-size <=2.0.2 contains infinite-loop DoS vulnerabilities in ICNS (GHSA-w3rx-r6r6-pgpr) and JXL/HEIF (GHSA-5p2g-fcmc-qvqq) parsers. The package is present in the dependency tree: @loaders.gl/textures -> texture-compressor -> image-size. Verification: texture-compressor main field is dist/cli/lib/index.js ? it is a CLI build tool, not a runtime library. @loaders.gl spawns it via child-process-proxy shim for build-time texture compression only. World Monitor has no web endpoint that accepts image uploads for texture compression; no attacker-controlled input reaches image-size at runtime. The vulnerability exists in the dependency tree and warrants a package override, but is not demonstrably exploitable via any HTTP request path in the current application.

Remediation

Add a package.json override to pin image-size to >=3.0.0: {"overrides": {"image-size": ">=3.0.0"}}. This resolves both GHSA-w3rx-r6r6-pgpr and GHSA-5p2g-fcmc-qvqq without requiring an @loaders.gl/textures update.

Evidence

[DEPENDENCY_RECORD] npm audit - image-size (GHSA-w3rx-r6r6-pgpr, GHSA-5p2g-fcmc-qvqq) - peer dep CLI, no web vector

```
npm audit reports image-size <=2.0.2 HIGH via @deck.gl -> @loaders.gl/textures -> texture-compressor -> image-size. Veri
... (truncated)
```

F03 Dependency: stream-json - O(depth^2) DoS via Deeply Nested JSON

LOW

FALS

Rejected

Location: package-lock.json:1

Description

FALSE POSITIVE. npm audit flagged stream-json via the chain @solana/web3.js -> jayson -> stream-json. All three packages are marked peer:true in package-lock.json ? peer deps are only installed when explicitly declared by the app. No World Monitor app code imports stream-json or jayson. The vulnerable O(depth^2) filter methods are never called in any app code path. This is a false positive from npm audit lockfile-based analysis.

Remediation

1. Upgrade stream-json to patched version. 2. Enforce JSON depth limits (max 20) before stream-json pipelines. 3. Audit call sites for user-controlled JSON input.

Evidence

[DEPENDENCY_RECORD] npm audit - stream-json O(depth^2) DoS (GHSA-528h-pc64-c93x)

```
npm audit:
stream-json Severity: moderate
Event loop blocked seconds-to-minutes on crafted deeply-nested JSON
https://github.com/advisories/GHSA-528h-pc64-c93x
```

F04 Dependency: uuid v3/v5/v6 - Missing Buffer Bounds Check on Custom Buf Argument

LOW

FALS

Rejected

Location: package-lock.json:1

Description

FALSE POSITIVE. uuid v8.3.2 is in the lockfile and would be installed, but GHSA-w5hq-g745-h8pq only affects uuid.v3(), uuid.v5(), and uuid.v6() when called with a custom buf argument of insufficient length. A complete search of all TypeScript and JavaScript source in World Monitor found zero calls to uuid.v3(), .v5(), or .v6() with any buf argument. Only standard v4() UUID generation is used. The vulnerable code path is not exercised by the application.

Remediation

1. Upgrade uuid to patched version. 2. Audit uuid.v3/v5/v6 call sites for external buf arguments accepted from user input.

Evidence

[DEPENDENCY_RECORD] npm audit - uuid missing bounds check (GHSA-w5hq-g745-h8pq)

```
npm audit:
uuid Severity: moderate
Missing buffer bounds check in uuid.v3/v5/v6 when buf is provided
https://github.com/advisories/GHSA-w5hq-g745-h8pq
```

F05 Informational: GitHub Actions CI Uses Default PostgreSQL Credentials

INFO

FALS

Rejected

Location: .github/workflows/test.yml:553

Description

FALSE POSITIVE (secrets scanner). The workflow file contains
UMAMI_TEST_DATABASE_URL=postgresql://postgres:postgres@127.0.0.1:5432/umami_integration. The string postgres:postgres is the official PostgreSQL Docker image factory default ? not a real secret. This is a CI-only ephemeral database bound to 127.0.0.1, destroyed after each CI run, with no access to any production system. The credential carries no secret value and matches the known-safe CI test pattern.

Remediation

1. Generate random credentials via GitHub Actions secrets.
2. Use `${{ secrets.CI_DB_PASSWORD }}` for ephemeral test databases.
3. Add comment documenting CI-only scope if static credentials are retained.

Evidence

[CONFIG_VALUE] Hardcoded postgres:postgres in .github/workflows/test.yml:553

```
.github/workflows/test.yml lines 551-558:
```

```
UMAMI_TEST_DATABASE_URL: postgresql://postgres:postgres@127.0.0.1:5432/umami_integration  
(used in two consecutive test steps)
```

Note: postgres:postgres are the official PostgreSQL Docker image default credentials.
CI-only ephemeral test database, not a production credential.

6. SECURITY POSTURE & CONCLUSION

Rating: MEDIUM RISK

Assessment conducted by ThreatLens Automated Security Platform

No significant security issues were detected. Maintain regular assessments.

NOTE: This report was generated by ThreatLens automated scanning. Results should be reviewed by a qualified security professional before making security decisions. Automated tools cannot detect every vulnerability type.